

Fast Feedback Resolution System (FFRS): A Five-Stage Pipeline from Capture to Close with an Agentic Respond Stage

Ganesh Butcha
ScaledAIOps
ganesh35@gmail.com

Work-in-progress draft, 7 September 2026 — not peer reviewed;
evaluation section pending the week-4 read-out (15 September 2026).
Comments welcome.

Abstract—Small software projects rarely close the loop with the people who report bugs or request features. Capture is easy; what follows—acknowledgement, a first substantive response, a decision, and notice of the outcome—is unowned, unmeasured, and therefore slow. This paper presents the Fast Feedback Resolution System (FFRS), a model that runs each feedback item through a five-stage pipeline (capture, acknowledge, route, respond, close) in which every stage is a timestamp recorded by the system that owns it. Its defining element is an *agentic Respond stage*: a scheduled AI agent produces the first substantive response to every item—for changes it can make, a complete pull request with tests and evidence for a developer to review and merge; otherwise a proposal that is executed only after the requester accepts it and a reviewer confirms it. No agent action takes effect without a human decision. Because the stages are timestamps, the metrics follow without manual bookkeeping. We describe a dependency-minimal, tool-agnostic reference architecture—a 4 KB (gzipped) widget, one serverless function, the project’s existing issue tracker as system of record, a private sidecar for the only personal datum, and a scheduled coding agent—and its open implementation, in production on scaledaiops.org since 18 August 2026, compared with aggregate data from an independent commercial deployment of a comparable widget. We report [TODO: n] items over [TODO: k] weeks (median time to first response [TODO: x], loop closure [TODO: y]%, agent share [TODO: z]%) against pre-registered targets and characterise when a minimal, human-reviewed agentic loop is sufficient. All artefacts are open; the metrics are recomputable from public data.

Index Terms—user feedback, feedback loops, AI agents, human-in-the-loop, issue tracking, DevOps metrics, software maintenance

I. INTRODUCTION

Every software project has a channel through which stakeholders raise feature requests (FRs) and bug reports (BRs). Far fewer have a working *loop*: an item that is registered, acknowledged, acted upon, and whose outcome is reported back to the person who raised it. In the author’s experience across nearly three decades of projects, the failure is seldom at capture—forms, mail, tickets and trackers abound—but in the interval that follows. Items wait for a triage meeting, for a developer with capacity, for a decision; the requester hears

nothing; and because that interval is unmeasured, its cost is invisible.

Two things have changed. First, the tooling that small teams already use—a Git host with an issue tracker, a serverless function, a mail API—is enough to record every stage of a feedback item as a timestamp, which makes response behaviour measurable for free. Second, AI coding agents can now read an issue, change code, run tests and open a pull request unattended. This makes it realistic to *assign the Respond stage to an agent by default* and to reserve human effort for review, acceptance and merge.

The Fast Feedback Resolution System (FFRS) is the author’s synthesis of these two observations into a small, prescriptive model: a pipeline with five stages—capture, acknowledge, route, respond, close—six timestamp-derived metrics, and one structural rule: the Respond stage is performed by a scheduled agent, with two paths. For items that require a code change, the agent produces a complete pull request—the fix, the tests it ran, and the evidence—for a developer to review and merge. For items that do not, the agent proposes a solution; the proposal is executed only after the *requester* accepts it and a *reviewer* confirms it. The loop closes when the outcome is recorded and reported back.

A. Contributions

- 1) **The FFRS model** (Sec. III): stages, the two-path agentic Respond stage with its human checkpoints, and metrics defined purely from timestamps.
- 2) **A tool-agnostic reference architecture** (Sec. IV) defined by roles and contracts—tracker, function, object store, mail, bot check, coding agent—in which the project’s existing issue tracker serves as the system of record, so that anyone with read access can recompute all metrics and each role can be filled by any equivalent product.
- 3) **An open implementation** (Sec. V): a 4 KB (gzipped) widget, one serverless function, and a scheduled agent, live on scaledaiops.org.
- 4) **An evaluation** (Sec. VI) on two sites against a pre-registered protocol, and an analysis of which parts of the design are load-bearing.

B. Research questions

- **RQ1** Can the FFRS model, including the agentic Respond stage, be implemented for a small project with no database and no third-party feedback vendor, and what does it cost to build and run?
- **RQ2** What response and closure behaviour does it produce in practice—time to first response, time to close, loop closure, signal ratio, and the share of items resolved by an agent-authored pull request?
- **RQ3** Which parts of the design are load-bearing for those outcomes, and which could be removed?

II. BACKGROUND AND MOTIVATION

A. The interval nobody owns

Consider the reference site of this paper, scaledaiops.org: a static, volunteer-maintained reference for an AI-operations framework, without accounts or analytics. Until August 2026 its only feedback path was a link to the project’s code-hosting organisation. A visitor without an account there could not report anything; one with an account had to leave the site, choose a repository and open an issue; nobody was notified; and no one could say how long such an issue typically waited. This is not unusual. Service desks and DevOps teams solve the same problem with people and process—ITIL-style acknowledgement, routing and resolution targets [1], or flow metrics such as lead time and time to restore [2]—but those presume staff whose job it is to work the queue.

B. What “small” means here

We use *small project* operationally: at most three maintainers, no dedicated support or triage role, and fewer than one hundred feedback items per month. At that scale a vendor feedback product is disproportionate (data leaves the project, cost per seat, another login), a self-hosted tracker is another system to operate, and the raw Git host is unfriendly to outsiders. FFRS targets exactly this gap.

C. Why an agent changes the economics

The scarce resource in a small project is maintainer attention. If the first substantive response to an item—a candidate fix with tests, or a concrete proposal—can be produced without it, attention is spent only on judgement: is the fix right, is the proposal acceptable. An independent commercial site that implemented a comparable capture widget (with a conventional, manual response process) reports the same pattern: capture works, items still stall. That observation motivated making the agentic stage a defining part of the model rather than an optional add-on.

The author’s own path—systems administration in the late 1990s and 2000s, web development in the 2010s, then DevOps and release management for large delivery organisations—crossed the same failure repeatedly: feedback was captured diligently and acted on late or never, and nobody could say how late. What was missing was never a form; it was an owner for the interval after the form, and a clock on it. FFRS is that owner and that clock, with the owner now an agent.

TABLE I
FFRS STAGES. “OWNER” IS THE SYSTEM WHOSE CLOCK RECORDS THE TIMESTAMP.

Stage	Definition	Owner / timestamp
Capture	The item exists durably with a public reference id	tracker: issue created
Acknowledge	Requester is told it was received (only if they asked to be)	mail: ack sent
Route	Item is in the working tool with kind/severity labels	tracker (may equal Capture)
Respond	First substantive response: an agent PR or proposal, or a human comment	tracker: first comment / PR
Close	Outcome recorded and reported to the requester	tracker: closed; mail: closing note

III. THE FFRS MODEL

A. Stages

The FFRS model treats feedback as a five-stage pipeline (Fig. 1). Each stage is a single timestamp, recorded by the system that naturally owns the event (Table I); FFRS prescribes no additional bookkeeping.

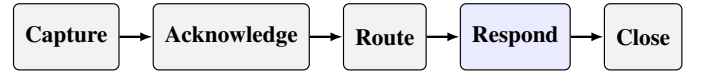


Fig. 1. The five-stage FFRS pipeline. The *Respond* stage is agentic by default.

B. The agentic Respond stage

Respond is performed by a scheduled agent that reads every open item not yet responded to. It follows one of two paths (Fig. 2):

a) *Code path*: If the item implies a code or content change the agent can make, it produces a *complete pull request*: the change, the tests it wrote or ran, their results, and a link back to the item. A developer reviews and merges (or rejects with a reason). Merge closes the item with outcome *fixed* or *shipped*.

b) *Proposal path*: Otherwise the agent posts a *proposal*: what it would do, what it needs, and any risk. Two human checkpoints follow: the *requester* accepts (the proposal solves their problem) and a *reviewer* confirms (it is safe and appropriate for the project). Only then does the agent execute and close the item; a rejection at either checkpoint returns it to the human queue. A proposal that receives no acceptance within a staleness window (we suggest 14 days) should likewise be returned to the human queue rather than left open; the reference deployment does not yet automate this.

The rule that makes FFRS “fast” is that the first substantive response is produced without waiting for a maintainer, and that maintainer effort is spent on judgement rather than production. The rule that keeps it safe is that nothing the agent produces takes effect without a human decision: merge, or accept-and-confirm.

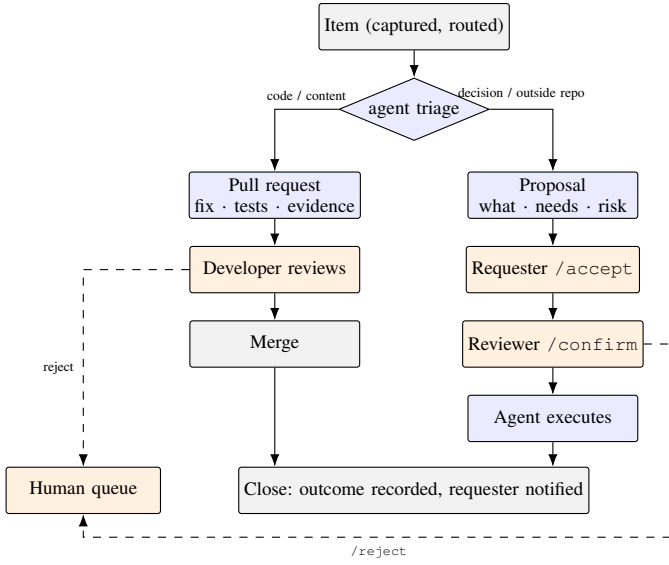


Fig. 2. The agentic Respond stage. Blue: produced by the agent; orange: human decisions; grey: system events. Nothing the agent produces takes effect without a merge or an accept-and-confirm.

C. Metrics Formalization

Let $\mathcal{I}$ be the set of feedback items in a given reporting window, partitioned by kind. For each item $i \in \mathcal{I}$, the pipeline records a tuple of monotonically increasing timestamps $\langle t_{\text{cap}}(i), t_{\text{ack}}(i), t_{\text{route}}(i), t_{\text{resp}}(i), t_{\text{close}}(i) \rangle$. For agent-driven systems, we additionally record $t_{\text{human}}(i)$, the timestamp of the first human decision.

All duration metrics are derived directly from these timestamps:

$$\text{TTFR}(i) = t_{\text{resp}}(i) - t_{\text{cap}}(i) \quad (1)$$

$$\text{TTHR}(i) = t_{\text{human}}(i) - t_{\text{cap}}(i) \quad (2)$$

$$\text{TTC}(i) = t_{\text{close}}(i) - t_{\text{cap}}(i) \quad (3)$$

Aggregates are reported as p50 and p90. Systemic rates are defined as follows, where $\mathcal{I}_{\text{closed}}$ is the subset of items that reached the Close stage:

$$\text{Loop Closure} = \frac{|\mathcal{I}_{\text{closed}}|}{|\mathcal{I}|} \quad (4)$$

$$\text{Signal Ratio} = 1 - \frac{|\mathcal{I}_{\text{spam}} \cup \mathcal{I}_{\text{dup}}|}{|\mathcal{I}|} \quad (5)$$

$$\text{Agent Share} = \frac{|\mathcal{I}_{\text{agent_merged}} \cup \mathcal{I}_{\text{agent_exec}}|}{|\mathcal{I}_{\text{closed}}|} \quad (6)$$

For items whose requesters opted in, we track the *consented notification rate* as the share for which the closing note was successfully sent. These definitions are deliberate analogues of flow metrics [2] and service-desk targets [1], reduced to what the system’s timeline can witness.

D. Design principles

- 1) *Durable first*. The item exists in the tracker before any notification or agent action.

TABLE II
ROLES IN THE REFERENCE ARCHITECTURE, THE COMPONENT USED IN THE REFERENCE DEPLOYMENT, AND EQUIVALENTS THAT SATISFY THE SAME CONTRACT.

Role	Reference	Equivalents
Issue tracker (system of record)	GitHub Issues	Jira, GitLab Issues, Linear, Azure Boards, Gitea
Capture function	AWS Lambda + API Gateway	Google Cloud Run / Functions, Azure Functions, Cloudflare Workers, a container behind any HTTP gateway
Edge / CDN path	CloudFront /api/*	Cloudflare, Fastly, or a reverse proxy in front of the site
Private object store	S3 (private prefix)	Google Cloud Storage, Azure Blob, MinIO
Secrets / kill switch	SSM Parameter Store	Secret Manager, Key Vault, Vault, environment configuration
Transactional mail	SES	SendGrid, Postmark, Mailgun, SMTP relay
Bot check	Cloudflare Turnstile	reCAPTCHA, hCaptcha, honeypot only
Coding agent	hosted coding-agent routine	GitHub Copilot coding agent, OpenHands, Aider or SWE-agent in a scheduled job, any agent that can open a pull request
Infrastructure code	as Terraform	OpenTofu, Pulumi, CloudFormation, Bicep

- 2) *Automate everything but judgement*. Ack, route, first response and closing notification are code; merge, accept and confirm are people.
- 3) *Measure timestamps, not opinions*. No status field is hand-maintained; metrics are recomputable by anyone with read access to the tracker.
- 4) *Responsible by default*. Personal data only with consent, stored apart from the public record, deletable; the agent never acts on production without a human checkpoint.
- 5) *Everything through version control*. Infrastructure as code and GitOps for every layer—site content, widget, function, cloud resources, the agent’s own instructions—so that each change, human or agent, is a reviewed commit and the audit trail is the commit log.

IV. REFERENCE ARCHITECTURE

Fig. 3 shows the reference architecture in terms of *roles*; Table II lists, for each role, the component used in the reference deployment and equivalent alternatives. FFRS prescribes the roles and the contracts between them, not the products: any tracker with issues, comments, labels and webhooks; any function-as-a-service runtime; any object store; any transactional mail API; any bot check; any coding agent that can open a pull request. The guiding constraint was that a small project should add *no* new system of record: the issue tracker it already uses *is* the store.

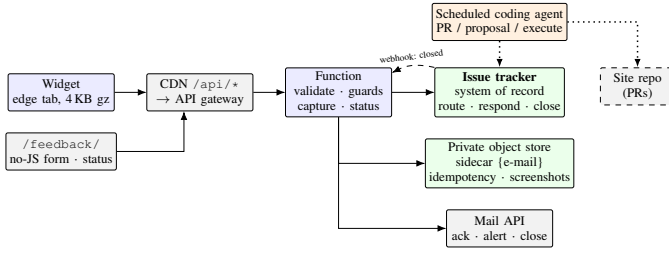


Fig. 3. Reference architecture. Solid: capture path; dashed: loop closure; dotted: agentic Respond stage. Blue: built for FFRS; green: stores (the tracker is the system of record); grey: existing platform services.

A. Capture surface

A framework-free widget (one script tag, configured by data attributes) renders an edge tab on every page. It offers “request a feature” and “report a bug”, captures the page URL and—for bugs—a client-side screenshot, and takes an optional e-mail with an explicit consent checkbox. A plain HTML form at `/feedback/` is the no-JavaScript fallback and the landing page for status links.

B. Tracker as system of record

The function validates the submission, applies guards (honeypot, per-address rate limit, a bot check), stores the screenshot in a private object store, and creates the issue synchronously with labels `kind:*`, `severity:*` and a hidden marker carrying the reference id. Only then does it return `202 {ref}`. If the tracker is unavailable the caller receives an honest `502` and the widget retries with the same idempotency key—nothing is half-stored. Route therefore coincides with Capture; Respond and Close are read from the tracker itself (first comment or pull request, close event, `outcome:*` labels or close reason).

C. Sidecar for personal data

The only personal datum, the requester’s e-mail, never enters the public record. A per-item JSON object in a private object-store prefix holds {e-mail, consent, screenshot key, ack sent, closing mail sent}. Deleting that object forgets the person; screenshots and idempotency keys expire automatically.

D. Loop closure

A tracker webhook fires on close; the function derives the outcome (`outcome:*` label $\succ$ close reason $\succ$ kind default) and sends the closing note. Comments and close time need no webhook—the tracker already records them. The same webhook receives new comments and removes the attribution footer that the agent’s hosting tooling appends to comments it posts, so the public record carries only the agent’s text and no product name.

E. Agentic Respond stage

A scheduled cloud agent runs against the tracker. For each open item without a first response, it either opens a pull request on the target repository (code path) or posts a proposal comment (proposal path). Acceptance and confirmation are

ordinary comments matching a small command grammar (`/accept`, `/confirm`, `/reject <reason>`) by the requester and by a maintainer respectively; the agent executes a proposal only after a maintainer’s confirmation (a requester without a tracker account cannot accept; the maintainer then acts for both). In the reference deployment the agent is a hosted coding-agent routine that runs hourly with the site and tracker repositories checked out and authenticated access to the tracker (Sec. V); any coding agent that can read the tracker, run the project’s tests and open a pull request fits the role. Its instructions are a versioned prompt of about 850 words that fixes the selection rule (open items with a kind label and no prior response, oldest first, at most three per run), the two paths, the label protocol, a 15-minute budget with at most five minutes on test infrastructure, and the rules *never push to main*, *never force-push a shared branch*, *one branch per item*, *clean up temporary files*, *never write an attribution trailer*. “Execute” for a static site means a pull request (content or code) or a drafted reply; the agent has no other write path. Its tracker identity is the maintainer’s, so agent comments carry a fixed marker and are excluded from the human metrics. A self-hosted CI-runner variant with the same protocol is kept as an alternative.

F. Security and threat model

An agent that reads untrusted input from the public internet and changes code is exposed to indirect prompt injection: a requester could embed instructions in a description that attempt to exfiltrate secrets or introduce harmful code. FFRS mitigates this structurally rather than by trusting the agent. The agent runs in an isolated sandbox that holds no production secrets (cloud credentials exist only in the infrastructure pipeline); its repository credentials allow it to push branches and open pull requests, while a repository rule blocks direct pushes to the main branch for every actor except a repository administrator; and the mandatory human checkpoint means that even a successfully injected payload cannot reach the mainline without a maintainer’s review and merge. The `ffrs/*` branch naming and the *never push to main* rule are additionally fixed in the versioned prompt, but the guarantee rests on the repository rule and the merge, not on the prompt.

G. Operability and audit trail

Every layer is declared in version control and changed only through it: the site and widget are built from a repository, the function is a versioned bundle, the cloud resources are infrastructure-as-code, and the agent’s protocol and prompt are files in the same repositories. Infrastructure changes go through `plan/apply` from a reviewed commit; content and code changes—including every change the agent proposes—arrive as pull requests. Consequently the audit trail for any question (*who changed what, when, and why; which items the agent touched; what a reviewer approved*) is the commit and pull-request history plus the public tracker, with no separate audit store to maintain. Three switches make FFRS detachable: a build-time flag (ships zero widget bytes), a runtime kill

TABLE III
REFERENCE IMPLEMENTATION (2026-08-18).

API (ffrs-api)	Node 22 Lambda, TypeScript strict, Zod at every boundary; 24 files, $\approx$ 900 LOC, 34 tests (in-memory tracker/store twins, no network); 416 KB bundle
Widget	10.9 KB (4.1 KB gz) + 5.8 KB CSS; no framework; screenshot library loaded on click
Infrastructure	Terraform module, 19 resources (function, API gateway, private bucket, mail identity, weekly rule, CDN behaviour); enable_ffrs=false plan is a verified no-op
Store	GitHub issue + one JSON sidecar per item; 0 tables
Agent	hosted coding-agent routine, hourly; $\approx$ 850-word versioned prompt (agent/routine-prompt.md)
Build effort	$\approx$ 1 working day across 8 phases (commit timeline in the repositories)
Run cost	within AWS free tier at the reference volume [TODO: confirm after 4 weeks]

switch in the parameter store (503 within a minute, no deploy), and an infrastructure flag that removes every resource; each is itself a committed change. Secrets are read at cold start from the parameter store and never appear in infrastructure state or in the repositories.

V. IMPLEMENTATION

Table III summarises the reference implementation—one instantiation of the roles in Table II, chosen because the project already ran on that cloud and that code host. All artefacts are public; commit hashes pin the versions evaluated. Everything product-specific is confined to two adapters (tracker, object store), the mail sender, and the deployment module; the domain logic, guards, widget and metrics do not depend on the products named here.

a) Engineering standards: TypeScript strict with schema validation at every boundary; persistence behind two ports with in-memory twins so the test suite needs no network; structured logs keyed by the item’s reference; formatting and validation before every infrastructure commit; small, single-purpose commits with imperative messages so the history reads as a change log. Infrastructure as code and GitOps are applied uniformly, making the audit trail in Sec. IV complete rather than partial.

b) Embedding contract: A site adopts FFRS with one tag:

```
<script src="/assets/js/ffrs-widget.js"
  data-endpoint="/api/feedback" data-site="example.
  org"
  data-turnstile="<sitekey>" data-screenshot="bug"
  defer></script>
```

and a function deployment configured by environment variables (tracker repository, mail sender, bucket). Everything site-specific is an attribute or a variable.

c) Reproducing the metrics: npm run metrics and npm run export in ffrs-api read the public tracker and emit per-week metrics and anonymised per-item rows (no body, e-mail or screenshot). The CSVs used in Sec. VI are committed under data/ in the paper repository.

VI. EVALUATION

A. Setting

Two deployments are studied. **S1** is the reference deployment on scaledaiops.org described in Secs. IV–V, in production since 18 August 2026 (T0); every item, comment, label and pull request is public. **S2** is an independent commercial site in the consumer-services domain that implemented and tested a comparable capture widget with a conventional, manual response process; it contributes anonymised aggregate data only—capture volume, item kinds and a time-to-close proxy derived from status changes—and, having no first-response timestamp, cannot report TTFR. S2 therefore serves as a comparison point for capture volume and closure, not for the agentic stage.

B. Protocol

The protocol was fixed in the project’s case-study document before data collection: a 12-week window from T0 with an interim read-out at week 4; per-kind, per-ISO-week metrics as defined in Sec. III; targets stated in advance—TTFR under two hours (the agent’s schedule), TTHR under 72 h at the median, TTC for bugs under 30 days at the median, closing notification for 100 % of consented closed items—and signal ratio and agent share reported without a target. The source of truth is the public tracker read by the published metrics/export commands; the resulting CSVs are committed with their export date. Items filed by the maintainer to test the system are labelled spam in the tracker and excluded.

C. Results at week 4

Table IV will report the interim metrics for S1 against the pre-registered targets, alongside the S2 baseline; the values are filled from the committed week-4 export (**[TODO: 2026-09-15]**) and are deliberately blank until then.

TABLE IV
INTERIM EVALUATION METRICS (WEEK 4). TARGETS ARE PRE-REGISTERED; S2 HAS NO FIRST-RESPONSE TIMESTAMP.

Metric	Target	S1 (bugs)	S1 (features)	S2 (all)
<i>n</i> (signal)	—	[TODO:]	[TODO:]	[TODO:]
TTFR (p50 / p90)	< 2 h	[TODO:]	[TODO:]	n/a
TTHR (p50)	< 72 h	[TODO:]	[TODO:]	n/a
TTC (p50)	< 30 d (bugs)	[TODO:]	[TODO:]	[TODO:]
Loop closure	—	[TODO:]	[TODO:]	[TODO:]
Consented notification	100 %	[TODO:]	[TODO:]	n/a
Signal ratio	—	[TODO:]	[TODO:]	[TODO:]
Agent share	—	[TODO:]	[TODO:]	0 %

TABLE V
TIMELINE OF THE FIRST PRODUCTION REQUEST (T0).

Time	Event
T+0:00	Item captured via widget (feature request).
T+1:10	First agent run starts (the routine was created that hour); by T+1:19 it has a correct fix but cannot push—the agent’s tracker identity lacked write permission on the organisation; it reports this precisely and stops.
T+1:21	Maintainer grants the permission.
T+1:55	Second agent run opens a pull request with the fix and its test results.
T+2:02	Maintainer reviews the one-line diff and merges within a minute; the item closes automatically with outcome <i>shipped</i> .

a) *First item (T0)*: The first genuine item on S1 was a feature request about the feedback widget’s own placeholder text, filed anonymously through the widget. Table V details the end-to-end timeline.

This sequence yielded a TTFR, TTHR, and TTC of approximately 2 hours, with an agent share of 100%. Operating cost (RQ1) will be reported from the actual cloud bill at week 4; the agent runs within a subscription and has no per-item API charge.

D. Ablation (RQ3)

The dependency-minimal architecture is the result of removing components from an earlier design; we argue for the load-bearing role of what remains:

- **Issue tracker vs. dedicated database**: the initial design had a relational store and an outbox. Removing them simplified the system and moved durability to the tracker; the trade-off is synchronous coupling—if the tracker is down, capture fails with an explicit 502 and the client retries safely with its idempotency key.
- **Sidecar vs. public personal data**: the private sidecar isolates e-mail addresses and screenshots; without it a public tracker could not be the system of record.
- **Bot check and honeypot**: protect the signal ratio by rejecting automated submissions before an issue is created; the counts of rejected and spam-labelled items are reported at week 4.

VII. DISCUSSION

A. When a minimal, agentic loop is sufficient

The reference deployment suggests three conditions under which FFRS as described is enough. First, the project already lives in one tracker: routing is then a label, respond and close are events the tracker records anyway, and there is nothing to synchronise. Second, most items are small: content corrections, wording, missing links, configuration—exactly the class an agent resolves end to end with a reviewable pull request. Third, maintainer attention is the binding constraint, not code-review capacity: the agent shifts effort from *producing* a first response to *judging* one, and judging a two-line diff takes a minute. Under these conditions the human role reduces to

merge, accept and confirm, and the loop closes without anyone owning a queue.

B. Where it breaks

The same conditions delimit the design. Items that need a decision the project has not made (scope, policy, roadmap) go down the proposal path and wait for a human exactly as before; the agent makes the waiting *visible* (a proposal with a timestamp) but not shorter. Requesters without a tracker account cannot accept a proposal, so the reviewer acts for both—acceptable for a small site, not for a product with many stakeholders. A project with several teams needs routing beyond a label. And an agent that opens pull requests is only as safe as the review culture around it: FFRS makes the review checkpoint structural, but a maintainer who merges without reading has removed it.

C. Removing the database was the right cut

The first design had a relational database, an outbox and a metrics view. Replacing them with the tracker as system of record removed one external service, one secret, one schedule and roughly a third of the code, and it made every metric recomputable by anyone with read access. What it cost was buffering: capture is now synchronous with the tracker, and an outage yields an honest 502 and a client-side retry rather than a queued item. For a small project this is the better trade—the tracker’s availability is already the project’s availability—and it keeps the whole system explainable in one diagram.

D. What the build taught us about agents in the loop

Two production-only findings shaped the agent’s operating envelope. The first agent run produced a correct fix within nine minutes and then failed on permissions: it could read the organisation’s repositories but not write, and it reported this precisely instead of retrying. The second run succeeded but spent most of its time on test infrastructure that did not match the sandbox. Both point to the same lesson: an agentic Respond stage needs a budget and an escape hatch (“state what you could not run and ship the pull request”) more than it needs cleverness, and the surrounding tooling—installable browsers, hermetic tests—matters as much as the prompt. Once both were in place, a real request was resolved by an agent pull request and a one-minute human review.

E. Cost

At the reference site’s volume the whole system runs within a cloud free tier and the agent within a subscription; the marginal cost of an item is the reviewer’s minute. We report actual figures in Sec. VI once the measurement window closes.

VIII. THREATS TO VALIDITY

a) *Internal validity*: The maintainer of the reference site is also the author of this paper and the reviewer in the loop; time to first human decision (TTHR) and time to close are therefore likely flattering relative to a project whose

maintainers did not build the system. We mitigate by pre-registering targets and the measurement protocol before data collection, by reporting per-item timelines rather than only aggregates, and by excluding items we filed ourselves to test the system (labelled `spam` in the public tracker, visible to anyone). The agent’s own responses are posted under the maintainer’s tracker identity; we distinguish them by a fixed marker so that TTHR counts only human comments, and we report TTFR (agent or human) separately.

b) External validity: The evaluation covers one small, niche, non-commercial site plus aggregate figures from one independent commercial deployment; feedback volume is low, so all rates are reported with n and no inferential statistics are claimed. The reference implementation targets one tracker and one cloud provider; the model depends on neither (Table II), and the tracker sits behind a four-method port, but a second adapter has not been built and exercised. Results for larger teams, higher volumes or regulated domains are out of scope.

c) Construct validity: “Respond” is operationalised as the first comment or pull request on the item, which may be a question or an incorrect attempt rather than an answer; “close” is the tracker’s close event with an inferred outcome, which measures that a decision was recorded, not that the requester was satisfied. Loop closure counts items whose close was recorded and consented notification whether the requester was told; neither is a satisfaction measure. Signal ratio depends on maintainers labelling spam consistently. Agent share counts merged agent pull requests and executed proposals; it says nothing about the effort a reviewer spent correcting them, which we report qualitatively only.

d) Reliability: All metrics are recomputable by anyone with read access to the public tracker using the published tooling; the CSV exports used in this paper are committed with their export dates. The dependence on the tracker cuts both ways: an outage or a change in the tracker’s API affects capture (an explicit 502 and client retry) and measurement, and comment timestamps are the tracker’s, not ours.

e) Ethical considerations: Personal data is limited to an optional e-mail stored with explicit consent, apart from the public record, and deletable; screenshots may capture on-screen content and are therefore private and expiring; the agent never acts on production without a human decision, and every action it takes is visible in the public tracker.

IX. RELATED WORK

a) Flow metrics and developer productivity: FFRS’s metrics are descendants of established flow vocabularies. The DevOps programme summarised in *Accelerate* [2] and later expanded in the SPACE framework [3] popularised outcome metrics—lead time for changes, deployment frequency—chosen because they can be measured mechanically. ITIL 4 [1] frames request handling as a sequence of acknowledgement, routing, and resolution targets. FFRS borrows the shape of both but reduces the scope to what one tracker can carry: no service desk, no separate measurement system. Where DORA measures the path from commit to production, FFRS measures

the path from stakeholder to outcome; where ITIL assumes a staffed queue, FFRS assumes an agent produces the first response.

b) Issue handling and maintainer fatigue: Large-scale mining studies established both the centrality and fragility of issue trackers. Bissyandé et al. [4] found that responsiveness varies widely, while Kalliamvakou et al. [5] catalogued the pitfalls of interpreting tracker data. Unmanaged queues are among the reasons open source projects fail [6]. FFRS takes these findings as motivation: the tracker is where decisions are made, so FFRS keeps it as the system of record and instruments it, assigning the first response to an agent to mitigate fatigue. Pull-based development made the pull request the unit of review [7] and its discussion the place where contributions are judged [8]; FFRS keeps exactly that as the human checkpoint on agent output.

c) Autonomous issue resolution: SWE-bench [9] made “resolve a real GitHub issue with a pull request” a benchmark task. SWE-agent [10], Agentless [11], ChatDev [12], and RepairAgent [13] have demonstrated that language models with purpose-built interfaces can resolve issues unattended at meaningful rates. FFRS does not advance agent capability. Instead, it positions the agent as the *default* owner of the Respond stage in a feedback loop, bounded by human checkpoints (merge for code; requester acceptance and reviewer confirmation for proposals), and it measures the result with the same timestamps as human-only handling. To our knowledge the combination—a feedback pipeline model, an agentic first response, and metrics recomputable from the public tracker—has not been described as a unit.

d) In-product feedback tooling and mining: Extracting features and bugs from user feedback is a rich area of research, with tools like SAFE [14] and various machine learning classifiers for app reviews [15], [16]. While commercial widgets (e.g., Sentry, Intercom) provide the capture surface, they rely on vendor-hosted storage. FFRS sits between these: an anonymous, account-free capture surface that funnels directly into the public tracker, avoiding proprietary databases while keeping personal data in a private sidecar.

X. CONCLUSION

Feedback loops in small software projects fail in the interval between “raised” and “acted on”, and they fail invisibly because that interval is unmeasured. FFRS makes the interval visible and hands its first half to an agent. The model is deliberately small: five stages, each a timestamp recorded by the system that owns it; six metrics that follow from those timestamps; and one structural rule—the Respond stage is performed by a scheduled agent, bounded by human checkpoints (merge for code; accept-and-confirm for proposals). The reference implementation adds no system of record beyond the tracker a project already has: a 4 KB (gzipped) widget, one serverless function, a private sidecar for the only personal datum, and a scheduled cloud coding agent. It was built in about a working day, is removable through a three-layer toggle, and has run in production on `scaledaiops.org` since

18 August 2026, where a real request has already been captured, resolved by an agent-authored pull request, reviewed in a minute, merged and closed.

We claim no more than the evidence supports: that a minimal, agentic feedback loop is *sufficient* for a small project under conditions we make explicit, and that its behaviour can be measured by anyone from public data. Future work is measurement over a longer window and more sites, a second tracker and runtime (for example a Jira-backed, container-hosted instance) to demonstrate the substitutability claimed in Table II, closure notifications that do not depend on e-mail, and a study of reviewer effort on agent-authored changes—the quantity that ultimately decides whether the human checkpoint stays a checkpoint or becomes the new bottleneck.

ACKNOWLEDGEMENTS

The FFRS concept and its design are the author’s. Parts of the implementation and of this manuscript were produced with an AI coding assistant under the author’s direction; all claims and data were verified by the author. The ScaledAIOps community hosts the reference deployment.

AVAILABILITY

Code (MIT): [Scaled-AIOps/ffrs-api](#); widget (CC BY-SA 4.0): [Scaled-AIOps/scaledaiops.org](#); infrastructure: [Scaled-AIOps/aiops-tf-infra](#). Feedback tracker (public data source): [Scaled-AIOps/feedback](#); paper source and data exports: [Scaled-AIOps/ffrs-paper](#). Metrics are reproducible with `npm run metrics` against that repository.

REFERENCES

- [1] AXELOS, *ITIL Foundation: ITIL 4 Edition*. The Stationery Office, 2019.
- [2] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.
- [3] N. Forsgren, M.-A. Storey, C. Maddila, T. Zimmermann, B. Houck, and J. Butler, “The SPACE of developer productivity,” *ACM Queue*, vol. 19, no. 1, pp. 20–48, 2021.
- [4] T. F. Bissyandé, D. Lo, L. Jiang, L. Réveillère, J. Klein, and Y. Le Traon, “Got issues? Who cares about it? A large scale investigation of issue trackers from GitHub,” in *Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE)*, 2013.
- [5] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, “The promises and perils of mining GitHub,” in *Proc. Working Conf. Mining Software Repositories (MSR)*, 2014.
- [6] J. Coelho and M. T. Valente, “Why modern open source projects fail,” in *Proc. 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2017.
- [7] G. Gousios, M. Pinzger, and A. van Deursen, “An exploratory study of the pull-based software development model,” in *Proc. 36th Int. Conf. Software Engineering (ICSE)*, 2014, pp. 345–355.
- [8] J. Tsay, L. Dabbish, and J. Herbsleb, “Let’s talk about it: Evaluating contributions through discussion in GitHub,” in *Proc. ACM SIGSOFT Int. Symp. Foundations of Software Engineering (FSE)*, 2014.
- [9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024.
- [10] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [11] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying LLM-based software engineering agents,” 2024.
- [12] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Jiang, Z. Liu, X. Huang, and M. Sun, “ChatDev: Communicative agents for software development,” 2023.
- [13] I. Bouzenia, P. Devanbu, and M. Pradel, “RepairAgent: An autonomous, LLM-based agent for program repair,” 2024.
- [14] T. Johann, C. Stanik, A. M. Alizadeh B., and W. Maalej, “SAFE: A simple approach for feature extraction from app descriptions and app reviews,” in *Proc. IEEE 25th Int. Requirements Engineering Conf. (RE)*, 2017.
- [15] S. Panichella, A. D. Sorbo, E. Guzman, C. A. Visaggio, G. Canfora, and H. C. Gall, “How can i improve my app? Classifying user reviews for software maintenance and evolution,” in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, 2015.
- [16] E. Guzman and W. Maalej, “How do users like this feature? A fine grained sentiment analysis of app reviews,” in *Proc. IEEE 22nd Int. Requirements Engineering Conf. (RE)*, 2014.